

Machine Learning

Train Test Validation- Week 9

Objectives

- Overfitting vs Underfitting
- Bias Variance Tradeoff
- Train test validation data
- Cross Validation

Slide credits:

Justin Pearson, Uppsala universitet

Troels C. Petersen, Niels Bohr Institutet

Where to find me?

Office:

Science Hall 250

Office Hours:

**Weds and Thurs 11:00 AM - 1:00 PM EST or by
appointment**

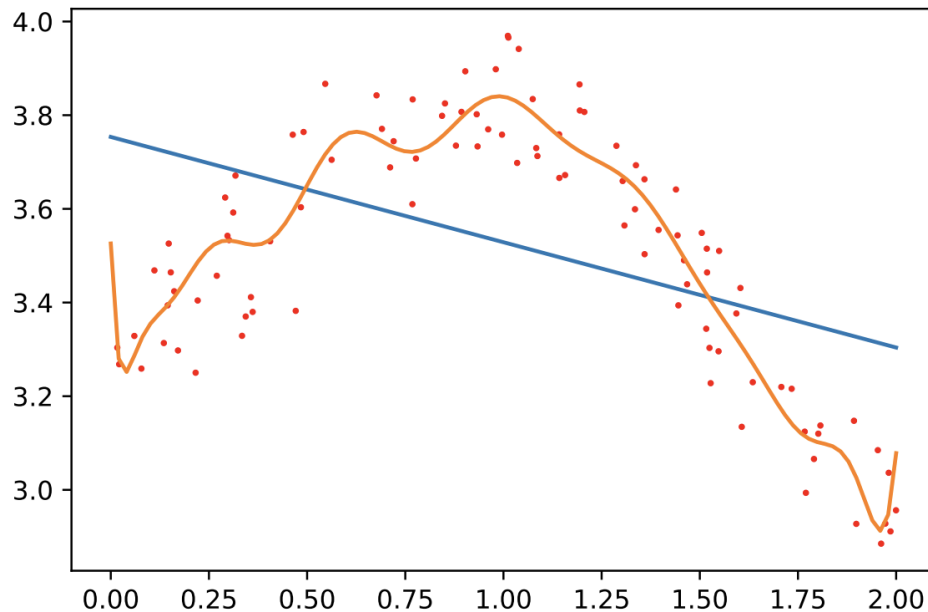
Overfitting vs Underfitting

Imagine we have some data in **x-y** plane.

The **blue** line and the **orange** curve represents two models that are fitting our data.

There are two observations:

1. **Blue** line is **under fitting** the data, or the model is **biased** towards solutions that will not explain the data.
2. **Orange** line is **overfitting** the data, or the model has a **high variance**. It is trying to model the irregularities in the data.



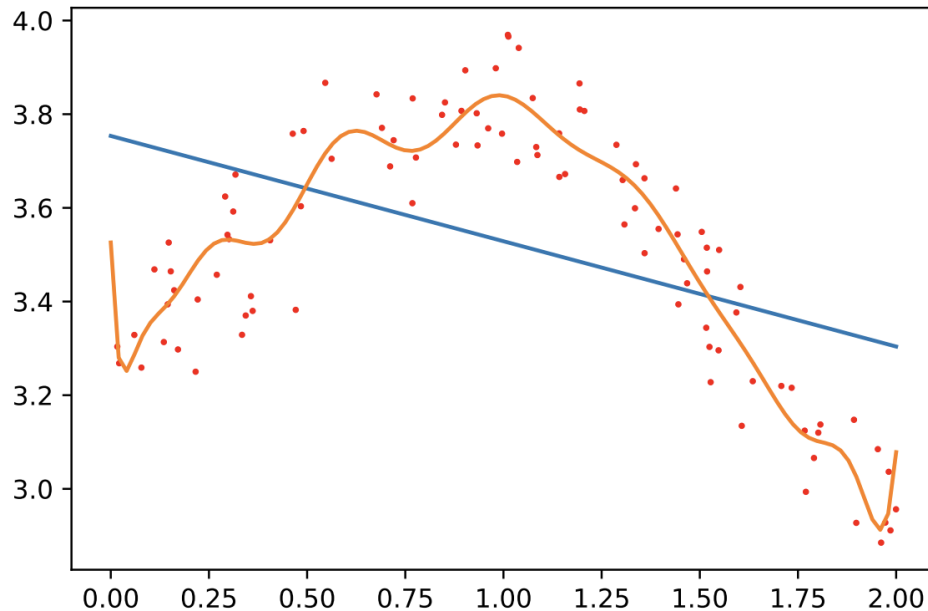
Overfitting vs Underfitting

The basic intuition behind **high bias** is that the model is **not sensitive enough** to the training data set in use.

If there's a slight change in the training data, the model will not change significantly.

The basic intuition behind **high variance** is that the predicted model is **too sensitive** to the training data set in use.

If there's a slight change in the training data, the model will change significantly.



Overfitting vs Underfitting

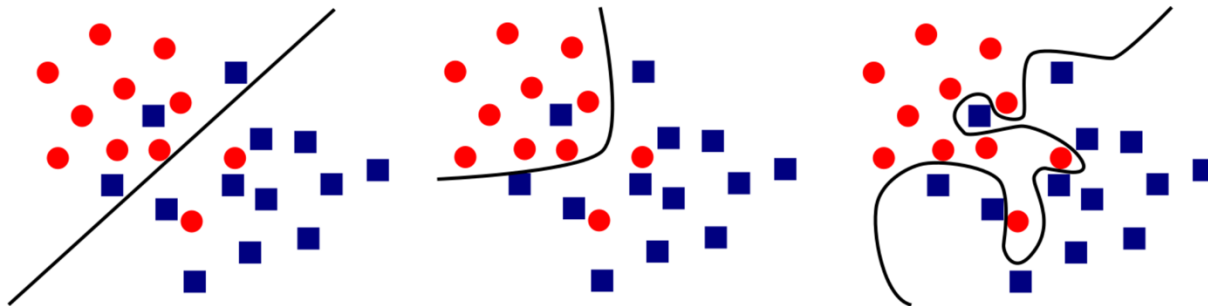


Figure: Under fitting, Just right and Over fitting in classification models

The following definitions are not really watertight and are intended to give you some intuition.

Model Bias : A model has high bias if the error/loss/cost function is high on the training data. It makes **bad prediction** on the training data.

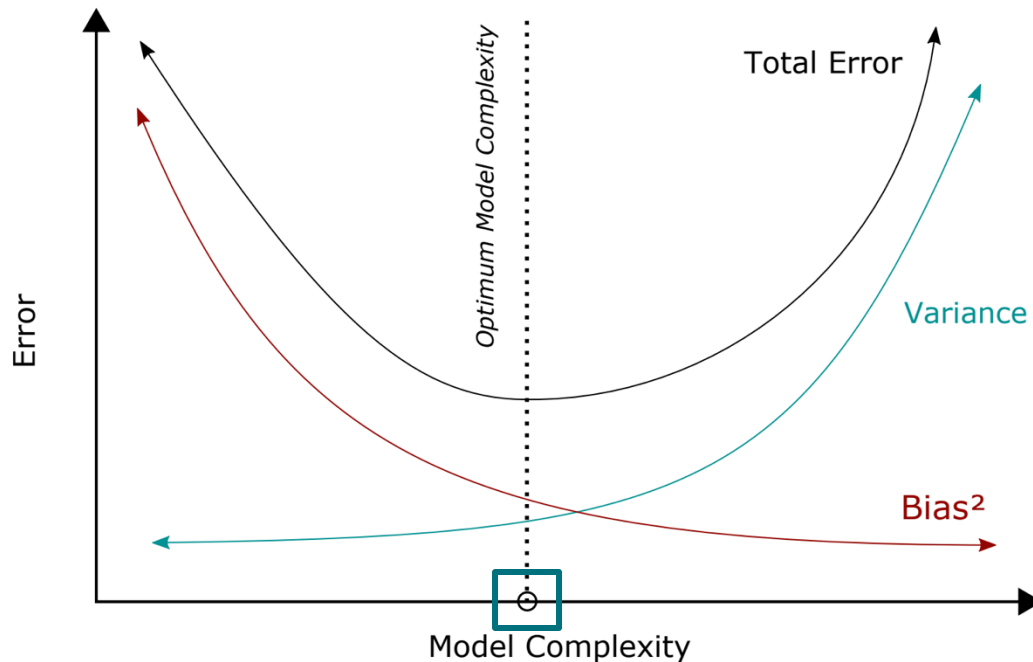
Model Variance: A model has high variance if the error/loss/cost function is high on the test data compared with the training data.

Bias Variance Tradeoff

You ideally want to find a **sweet spot**.

This graph is true for traditional machine learning methods and statistics.

Even without going into the mathematics you will get quite far, if you think about the complexity in your model.



Train Test Validation

If you train you algorithm on all data, you will not recognize overfitting, nor what the expected performance on new data will be. Therefore, we divide the data into:

Train Dataset

- Set of data used for learning (by the model), that is, to fit the parameters to the machine learning model using stochastic gradient descent.

Validation Dataset

- Set of data used to provide an unbiased evaluation of intermediate models fitted on the training dataset while tuning model parameters and hyperparameters, and also for selecting input features.

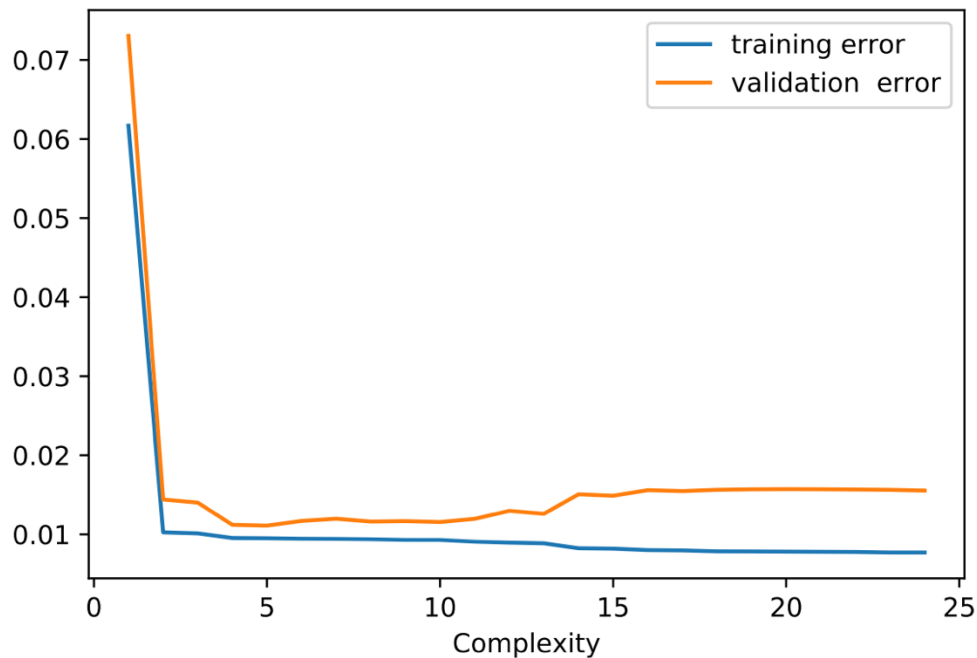
Test Dataset

- Set of data used to provide an unbiased evaluation of a final model fitted on the training dataset.



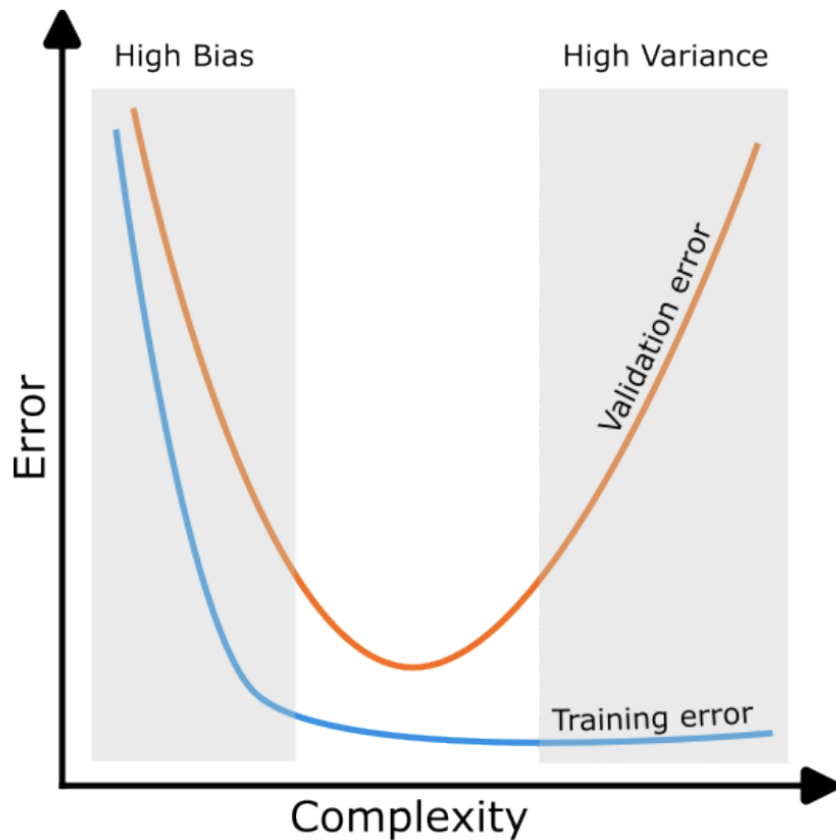
Train Test Validation

Assuming that you have split the model into training and validation sets then you can look at training and validation errors as your models get more complicated. → **Elbow method**



Train Test Validation

- If the test set error and the training set error is **very high**, then you are probably under-fitting.
- When the training error gets smaller and smaller, but your **test set error** starts increasing you are probably overfitting.



How do we split the data then...

You can obviously write your own code to split the data or use Sklearn's inbuilt functions:

Scikit-Learn method:

```
from sklearn.model_selection import train_test_split
X_train, X_rem, y_train, y_rem = train_test_split(X, y, train_size=0.8)
X_valid, X_test, y_valid, y_test = train_test_split(X_rem, y_rem, test_size=0.5)
```

There are a few important things to remember:

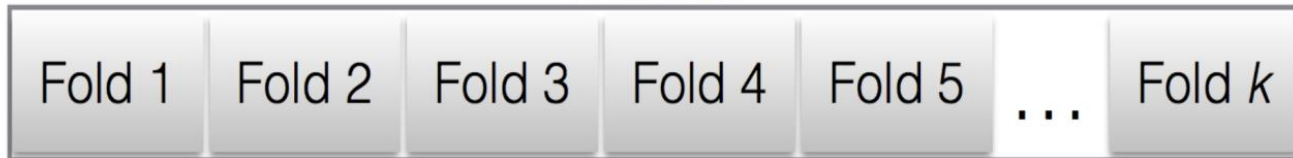
- Always do the data cleaning, selecting, weighting, etc. before splitting!
- If there is “more than enough” data, then use less than the total.
- If there is “a little too little” data, then use cross validation (next)

K-fold Cross Validation

In case your **data set is not that large**, one can train on most of it, and then test on the remaining $1/k$ fraction. This is then repeated for each fold...

K-fold cross validation is CPU-intensive, but easily parallelizable and smart especially for small data samples.

Dataset

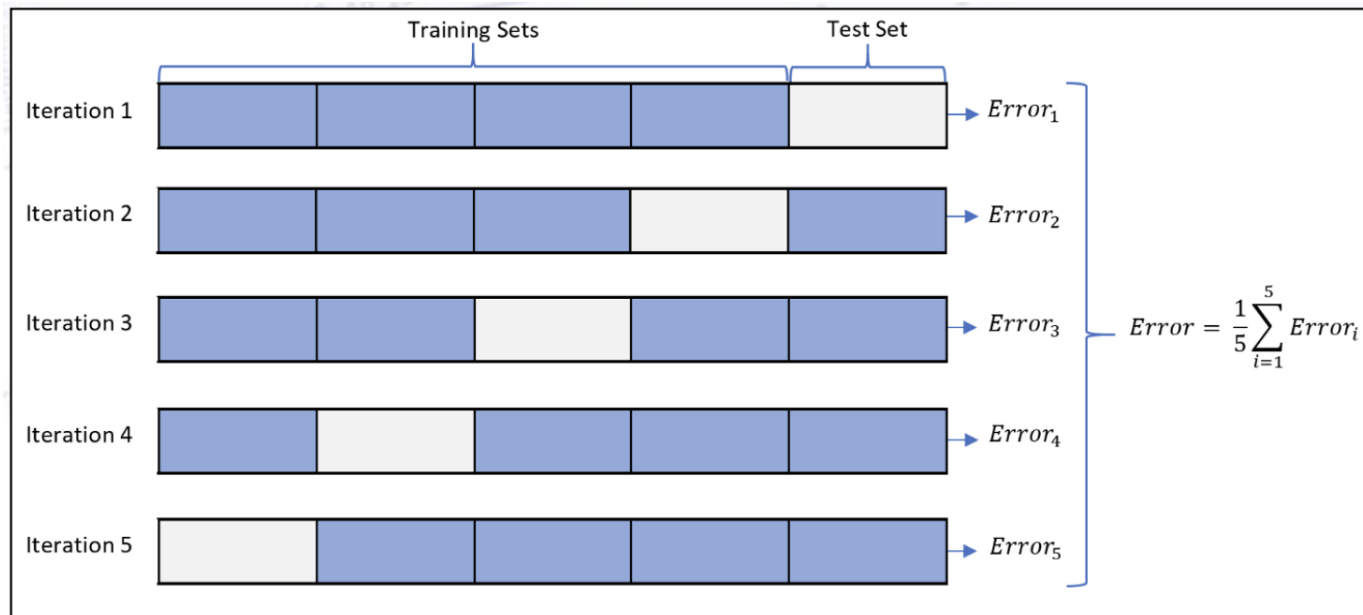


- ▶ Split the dataset into k randomly sampled independent subsets (folds).
- ▶ Train classifier with $k-1$ folds and test with remaining fold.
- ▶ Repeat k times.

K-fold Cross Validation

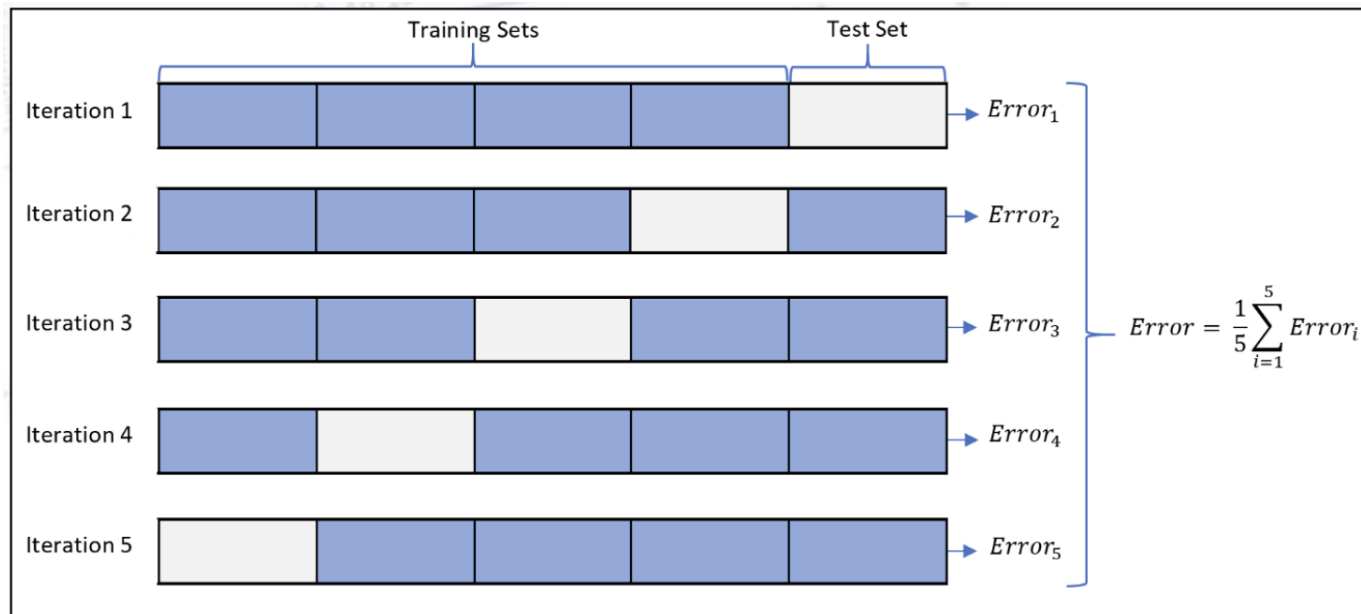
The k-fold cross validation (CV) does not only allow you to train on almost all and test on all the data, but also has a two additional advantages:

- If you consider the performance (“Error” below) on each fold, then you can also calculate the uncertainty on the performance.



K-fold Cross Validation

- Since you can test on all data, the uncertainty on the loss estimate goes down.



When to use K-fold Cross Validation

Note that cross validation applies to mostly three cases:

- When there is little test data.
- When you want uncertainty on performance
- When accurate performance measure is required i.e. find the best model out of many.

ANY QUESTIONS?